

PRUEBA DE SOFTWARE

Errores: Errores humanos. Tipográficos o sintácticos.

Defecto: Condiciones impropias de un programa que generalmente son el resultado de un error. Comentarios incorrectos o causas ajenas al desarrollador, como problemas con las herramientas. (Provoca bugs o bugs latentes). No todos los errores producen defectos en el programa (por ejemplo, errores de documentación).

Bugs: Defecto del programa que se encuentran operando el mismo, durante su prueba o durante su uso. Bugs provienen de los defectos. pero no todos los defectos causan bugs.

Fallas: Mal funcionamiento en una instalación de usuario. Puede ser provocado por un bug, mala instalación de un programa, falla en el hardware, etc. Eventos relacionados con el sistema

Problemas: Dificultades encontradas por los usuarios. Puede provenir de falla, mal uso o mal entendimiento. Eventos relacionados con los humanos. Errores de operador. Eventos relacionados con el sistema

Prueba genérica: Cualquier actividad dirigida a evaluar un atributo o capacidad de un producto de soft y determinar si alcanza los resultados requeridos.

Prueba dinámica (testeo): Proceso de ejecutar un programa en su totalidad o en parte, con la intención de encontrar bugs. Prueba de código.

Prueba estática (revisión): Revisar o analizar un producto con el objeto de identificar defectos y mejoras. Pruebas de documentación. No hay ejecuciones de prueba ni código.

Debugging: Diagnosticar la causa de un bug y corregirlo. Es una **corrección**, no prueba.

Tipos de pruebas genéricas:

- Estáticas: Corroborar que no haya información repetida, no sea ambigua, no contradictoria, que sea clara.
 - Tiempo de carga negativa: la web carga mas rápido de lo esperado.
 - Revisión de código fuente: código fuente comprensible para futuro mantenimiento. Legible. Buenas prácticas de código limpio. No código malicioso. Se aplican estándares del lenguaje. Ej.: indentaciones, régimen en cómo se declaran las variables y con nombres identificables.
 - Solo se evalúa el producto, no quien lo desarrollo.
- Dinámicas:
 - Pruebas de estrés
 - Pruebas de volumen: Como funciona el soft con X cantidad de datos.
 - Pruebas de performance: Se evalúa tiempo. Con X cantidad de volumen, responde en Y cantidad de tiempo.
 - Pruebas de seguridad: Pruebas para validar la vulnerabilidad del software.
 - Pruebas unitarias: Suelen ser los únicos test que hace el desarrollador. Pruebas de caja blanca.
 - Pruebas de usuario/Pruebas de aceptación: Se le muestra el desarrollo al usuario.
 - Smoke test: Pruebas básicos antes de comenzar el test.

Verificación: Que funcione.

Verificación: Si es lo que el usuario quería.

Pruebas dinámicas (Testeos - se ejecuta el código):

- Unidad: Se testea las unidades mínimas del sistema. Testeo mínimo de una parte específica de un código. Se testea una unidad específica de un sistema mayor. La única que suele testear el mismo autor.
- Integración: Se testea cuando se integran/unifican varias partes del sistema, como funcionan juntas
- Sistema: Se testea cuando el sistema está terminado. El sistema en conjunto.
- Aceptación: Pruebas del usuario o con el usuario. Donde acepta la funcionalidad. Confirma si eso era lo que se quería o no.
- Regresión: Verificar que se corrigieron los defectos y no se introdujeron nuevos

Pruebas estáticas (revisiones – no se ejecuta el código):

- Revisiones gerenciales: Desarrollo del proyecto. Se ven temas de recursos, tiempo, dinero.
- Revisiones técnicas: Revisar aspectos técnicos de un sistema, de una arquitectura (si es adecuada, etc)
- Inspecciones: Revisión de código con el equipo, reunión de orientación, informar formalmente los resultados, cumpla las normas
- Walkthroughs: Revisión informal de la documentación o el código con alguien del equipo. Es simplemente una reunión del dev y el tester por ej, para revisar una historia de usuario o funcionalidades.

Testeo caja negra: Requerimientos funcionales.

Testeo caja blanca: Lógica interna y estructura del programa.

Pruebas estáticas:

Objetivos:

- Encontrar defectos lo más temprano del ciclo de desarrollo
- Asegurar que se llegue a un acuerdo técnico sobre el producto
- Verificar que cumpla los criterios predefinidos
- Completar formalmente las tareas técnicas
- Proveer datos del producto y del proceso de revisión

Beneficios:

- Asegurar que el equipo este al tanto con los detalles técnicos del producto
- Ayudar a formar un equipo técnico eficiente
- Ayudar a utilizar los mejores talentos de la organización
- Proveer al team sentido de pertenencia y compromiso
- Ayudar al team a desarrollar habilidades de revisión

Principios básicos:

- Revisar formal y estructuradamente con un sistema de listas de verificación y roles definidos
- Las listas de verificación y estándares se desarrollan para cada tipo de revisión y se adaptan a las necesidades de cada proyecto
- Los revisores preparan e identifican inquietudes y preguntas antes que comience la revisión
- El objetivo de la revisión es **identificar defectos**, no resolverlos
- Solo se informan los resultados de las revisiones a los niveles superiores, ellos no las realizan ni supervisan
- La revisión genera información para controlar la calidad del producto, y de esta forma monitorear y mejorar el proceso de revisión

Roles de los participantes:

- Moderador: Lidera la revisión y asegura se alcancen los objetivos
- Productor/es: Responsables de que el producto se revise
- Revisores: Quienes están interesadas y que están al tanto que el producto se revise
- Escriba: Quien registra los resultados significativos de la revisión

Fases:

- **Planificación:** En la revisión, se elige quienes participan, sus roles y se les brinda copias de la documentación con la lista de verificaciones. Se determina:
 - Objetivos de la revisión
 - Criterios de finalización de la revisión
 - Lugar y fecha de revisión
 - Agenda de la reunion
- **Orientación inicial:** Reunión opcional, previa a la revisión, para dar al team una idea del producto a revisar
- **Preparación individual:** El revisor debe analizar la documentación recibida e identificar inquietudes, defectos y preguntas.
- **Reunion de revisión:** El productor presenta una visión del producto. Los defectos que se detecten se identifican, se registran y se les asigna el nivel de severidad.
Se presentan 3 posibles situaciones:
 - No tiene defectos: se cierra revisión
 - Defectos menores: Se continua con seguimiento y evaluación
 - Defectos graves: Se debe corregir los defectos e iniciar un nuevo proceso de revisión
- **Seguimiento:** El productor corrige los defectos identificados y genera un informe con las acciones correctivas realizadas
- **Evaluación:** Se analiza el informe presentado por el productor, de forma tal de determinar si se han corregido los defectos y no se ha introducido nuevos

	Revisiones Gerenciales	Revisiones Tecnicas	Inspecciones	Walkthroughs
Objetivo	Asegurar el progreso del proyecto, usar correctamente los recursos y recomendar acciones preventivas	Evaluar la conformidad de un producto respecto a especificaciones y asegurar la integridad de los cambios	Detectar/identificar defectos y verificar su correccion	Detectar defectos, examinar alternativas y generar un foto de aprendizaje
Grado de formalidad	Medio	Medio	Alto	Bajo
Participantes	Responsable del proyecto, lideres tecnicos e ingenieros	Lideres tecnicos e ingenieros	Ingenieros pares del responsable de proyecto	Colegas del responsable del producto
Liderazgo	Responsable del proyecto	Lider tecnico	Moderador	Productor
Verificacion de cambios y correcciones	Se deja para otros puntos de control	El lider tecnico debe verificar los cambios que resulten necesarios	Es obligatoria	Se deja para otros puntos de control
Extras:	Las decisiones se toman en la reunion y/o como resultado de recomendaciones	Las discrepancias son investigadas para confirmarlas	Los defectos deben ser removidos	El productor decide si realizar cambios y correcciones

Pruebas dinámicas

Clasificación de pruebas

- Estáticas (Revisiones)
 - Gerenciales
 - Técnicas
 - Inspecciones
 - Walkthroughs
- Dinámicas (Testeos)
 - Unidad
 - Integración
 - Sistema
 - Aceptación
 - Regresión

Definiciones:

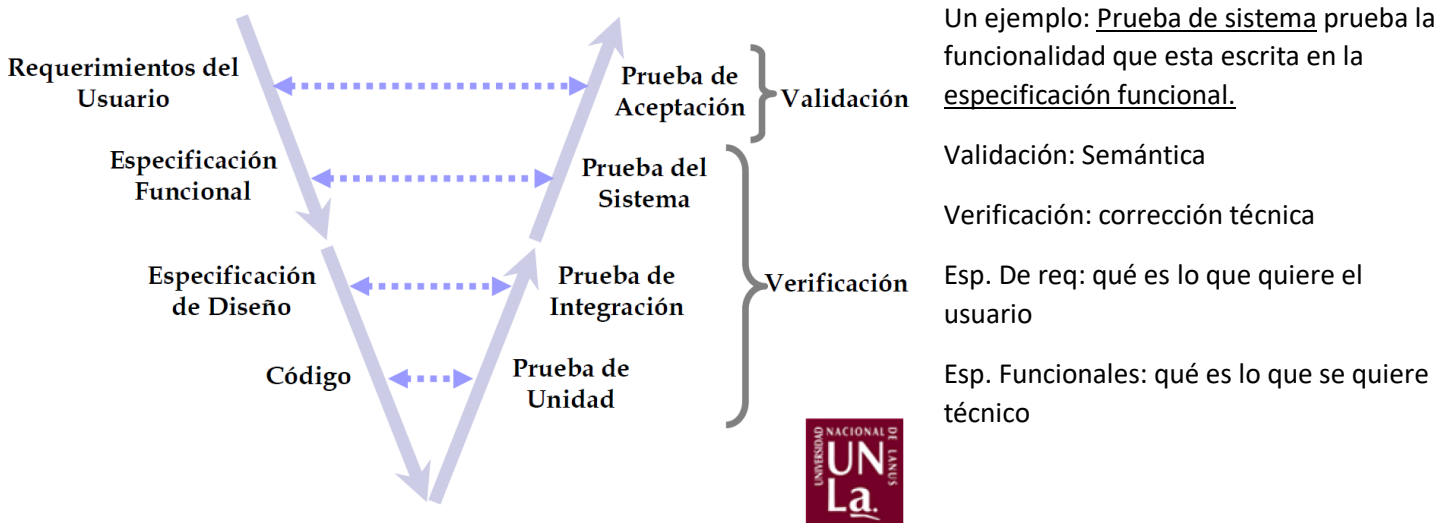
- Verificación: Intento de encontrar bugs, ejecutando un programa en un ambiente simulado o de testeo. Es el proceso que provee la corrección del programa.
- Corrección: Un producto es correcto si posee una sintaxis adecuada. Indica si se está desarrollando el producto correctamente.

- Validación: Intento de encontrar bugs, ejecutando un programa en un ambiente real. Es el proceso que provee la validez del programa.
- Validez: Un producto es válido si responde a una semántica adecuada. Indica si se está desarrollando el producto correcto.

Principios básicos

- El objetivo del testeo es descubrir bugs.
- Un buen caso de prueba es aquel que tiene altas probabilidades de detectar un bug.
- Una parte necesaria de todo caso de prueba es la descripción del resultado esperado.
- Los casos de prueba son para condiciones/entradas válidas e inválidas.
- Se debe evitar el testeo no reproducible o “al voleo”.
- Un programador debe evitar testear su propio programa.
- El test completo es casi imposible.
- El test es una actividad extremadamente creativa, intelectual y difícil.

Marco del testeo - Modelo V



Pruebas de integración: Se prueba la integración de varios softwares

Prueba de sistema: Se prueban especificaciones técnicas en el front del desarrollo (Pruebas de performance, de volumen, seguridad)

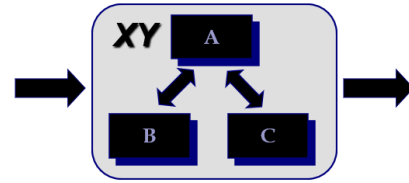
Pruebas unitarias: Se prueba el código. Código maligno y lógica del back.

Método de testeo: (Técnicas que se utilizan para generar casos de prueba)

- Caja blanca – como esta codificado
- Caja negra – conjunto de valores posibles, solo se ve las entradas y salidas

En **CAJA BLANCA** podemos validar código malicioso.

- Para derivar casos de prueba se examina la estructura y la lógica interna del programa.
- Comprende las técnicas de:
 - Prueba de caminos posibles.
 - Prueba de estructuras de datos.
 - Prueba de decisiones y estructuras lógicas.



- Gráficamente, se tiene la siguiente visión:

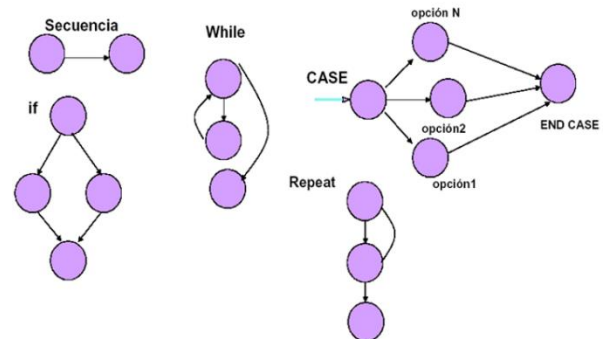
Grafo de complejidad ciclomática

- Es una medida del software que aporta una valoración cuantitativa de la complejidad lógica de un programa.
- Dentro del contexto del método de pruebas del camino básico define el número de caminos independientes de un programa.
- Por camino independiente se entiende aquel que introduce un nuevo conjunto de sentencias o una nueva condición. En términos del grafo, por una arista que no haya sido recorrida antes.
- Nos da una cota o límite superior para el número de casos de prueba.
- Complejidad logarítmica que tiene el código. La complejidad es por cada método. $CC = \text{Aristas} - \text{Nodos} + 2P$ (puertas: entrada y salida)

Cubrimientos:

Sentencias:

- Esta cobertura requiere que se ejecute por lo menos una vez cada sentencia del programa.
- Este criterio es necesario, pero no suficiente
- Sentencias: while, for, catch, if, else, etc

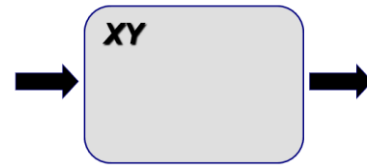


Condiciones

- En este criterio es necesario presentar un número suficiente de casos de prueba de modo que cada condición en una decisión tenga, al menos una vez, todos los resultados posibles.
- Este criterio es más fuerte que el anterior.
- Condiciones: Recorrer las sentencias dependiendo la condición (OR/AND). Se deben cubrir todas las posibilidades. Ej: if (condición1 or condición2), debe cubrir todas las alternativas
- If(true or false)
- If(false or true)
- If(true or true)
- If(false or false)

CAJA NEGRA

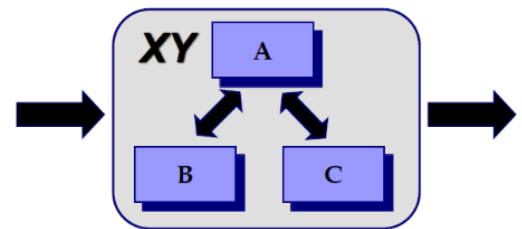
- Para derivar casos de prueba se examinan los requerimientos funcionales del programa.
- Comprende las técnicas de:
 - Prueba de valores límite.
 - Particiones de equivalencia.
 - Prueba por comparación.
- Gráficamente, se tiene la siguiente visión:



Tipos de pruebas dinámicas

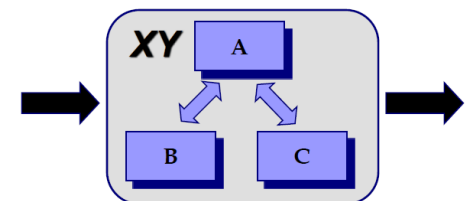
Pruebas de unidad

- Verifican programas o módulos individuales.
- Son ejecutadas, típicamente, en ambientes aislados o especiales.
- Generalmente son ejecutadas por la misma persona que programó el módulo o programa.
- Son los tests que suele detectar la mayor cantidad de bugs.
- Gráficamente:



Pruebas de integración

- Verifican las interfaces entre partes de un sistema (módulos, componentes o subsistemas).
- La integración puede ser total (Big Bang) o gradual:
 - Top-Down: Se necesitan "stubs" para simular módulos inferiores.
 - Bottom-Up: Se necesitan "drivers" para simular módulos superiores.
- Gráficamente:

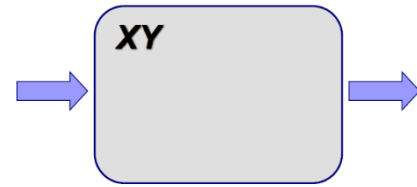


Pruebas de sistema

- Verifican el sistema global contra sus objetivos iniciales.
- Además, se debería testear, entre otros:

- Volumen (Load).
- Instalabilidad.
- Operabilidad.
- Seguridad.
- Performance (Stress).

- Gráficamente:



Pruebas de aceptación

- Validan el sistema contra los requerimientos del usuario.
- Aunque no siempre, son ejecutadas, típicamente, en el ambiente real del usuario.
- Los casos de prueba son generalmente especificados y ejecutados por los mismos usuarios.

Pruebas de regresión

- Su nombre se debe a que se contrapone con las demás pruebas dinámicas, que son progresivas (testeo de nuevas funciones y características).
- Son la ejecución de un subconjunto de casos de prueba, previamente ejecutados, para asegurar que los cambios a un programa o sistema no lo degradan.
- Uno de los desafíos es la selección de los casos de prueba que se deben volver a ejecutar.
- Es el test más común en la etapa de mantenimiento de un sistema.
- Las pruebas de regresión son siempre una parte integrante de las pruebas dinámicas progresivas.

Otras pruebas

- Smoke testing
- Alpha testing
- Beta testing

Fases

Planificación: Se define el plan de pruebas y los objetivos de la misma. Quienes hacen las pruebas, qué pruebas y qué módulos, plazos de fechas, criterios de aprobación de cada prueba, criterios de severidad, errores estéticos. Se debe especificar, entre otros:

- Funciones, procesos y características a testear y a no testear.
- Métodos, tipos y orden del testeo.
- Criterios de aprobación de la prueba.
- Criterio de clasificación de severidad de bugs.
- Ambientes, recursos físicos y herramientas a utilizar.
- Recursos humanos, responsabilidades y cronograma.

Diseño: Se determina cómo cumplir con los objetivos establecidos en la planificación. Qué elementos agrego a la prueba. Se debe especificar, entre otros:

- Condiciones generales a testear e ítems generales a verificar por función o proceso.
- Procedimientos de ejecución de casos de prueba.

Está compuesto por:

- Casos: ID. Función, condición, precondiciones, resultados esperados.
- Ejecución: Resultado obtenido, fecha, tester, versión

Derivación de casos: Es la especificación de cada uno de los casos de prueba. Para cada caso se debe especificar, entre otros:

- Identificador.
- Función a testear.
- Condiciones a testear.
- Resultado esperado.

Preparación: Es la confección de todos los elementos necesarios para la ejecución de la prueba, tales como:

- Archivos.
- Scripts.
- Formularios.
- Tarjetas.

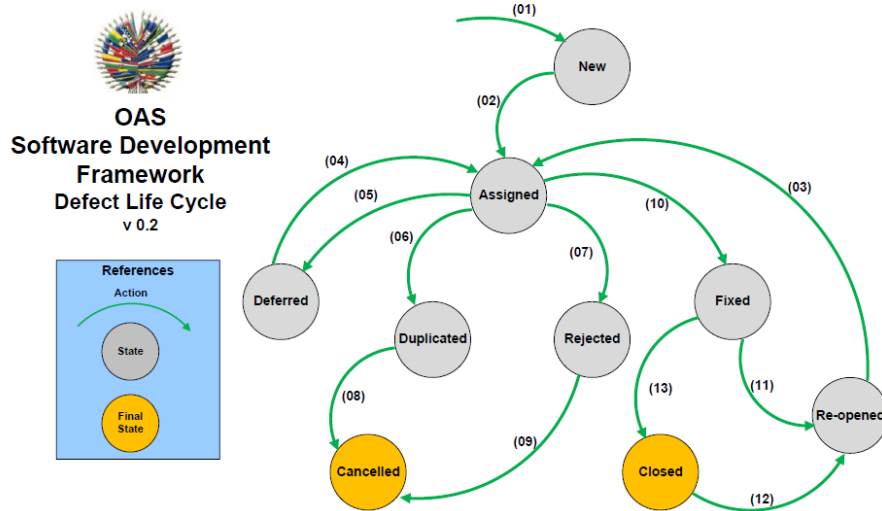
(Información o datos necesarios para testear, permisos, accesos, etc.)

Ejecución: Es la ejecución de cada caso de prueba. Por cada caso se debe registrar, entre otros:

- Resultado obtenido.
- Fecha, hora y responsable de la ejecución.
- Capturas de las evidencias

Análisis y evaluación: Se analizan los resultados obtenidos y en base a los criterios establecidos en la planificación. Se debe determinar, entre otros:

- Condiciones de suceso de cada bug detectado.
- Severidad de cada bug.
- Aprobación o no de la prueba.
- Porcentaje de casos ejecutado



- 1 – Se detecta el defecto con severidad
- 2 – Se asigna al desarrollador
- 3 – Se puede diferir el defecto ya que se resolverá en otra versión.
- 4 – Si el defecto está duplicado se cancela y se deja el original
- 5 – Se puede rechazar el defecto por que la “falla” es quizás algo que realmente debe pasar
- 6 – El defecto se arregla
- 7 – Se cierra (solo el tester) ya que da el ok de que fue resuelto
- 8 – Se puede reabrir dado que volvió a fallar en nuevos sprints
- 9 – Si se re-abre un defecto, se vuelve al flujo desde asignar

State	Action	Action responsible
New	(1) A defect is posted into the bug tracker tool and various fields are assigned such as Description, Detection date, Severity level, Version in which was detected, System module in which was detected, System Status (in Production or in Development).	- SQA team - Project Manager
Assigned	(2) The review team reviews each new defect and assigns various fields such as Owner, Severity, Priority, and Version to fix it in. Then it gets assigned to the corresponding developer to fix it or to the Project Manager for further analysis. (3) A reopened defect is assigned to the corresponding developer to fix it. (4) A deferred defect is assigned to the corresponding developer to fix it.	- Review team - Project Manager
Deferred	(5) The defect is expected to be fixed in next releases, is low priority, there is insufficient time for the release, or it may not have major effect on the software.	Project Manager
Duplicated	(6) It is detected that the defect was already reported.	Development team
Rejected	(7) It is not considered that the defect is valid due to one of following reasons: a) Not able to reproduce b) Not a valid defect and it is as per requirement c) Test data used was invalid d) Defect referring to the requirement has been de-scoped from the current release, tester was not aware of these late changes.	Development team
Cancelled	(8) Tester realized that the defect logged by him was invalid and agreed to cancel it. (9) Tester realized that the defect logged by him was duplicated and agreed to cancel it.	SQA team
Fixed	(10) Assigned developer has fixed the defect and has unit tested the fix. The code changes are deployed in test environment for verifying the defect fix.	Development team
Re-opened	(11) Defect is re-tested but it is not fixed as expected, is not working, or is partially fixed. (12) A previously closed defect is reopened because problem persists.	SQA team
Closed	(13) Defect is re-tested and it has been fixed or, as an exception, defect is closed under Project Manager indication.	SQA team

Métricas de pruebas

Id	Descripción	Tipo
CCPE	Cantidad de casos de prueba ejecutados	Cuantitativo
PCPE	Porcentaje de casos de prueba ejecutados.	Cuantitativo
CDE	Cantidad de defectos encontrados.	Cuantitativo
PDES	Porcentaje de defectos de encontrados por severidad.	Cuantitativo
PCS	Porcentaje de cubrimiento de sentencias.	Cuantitativo
PCC	Porcentaje de cubrimiento de condiciones.	Cuantitativo

Medidas – Cantidad de líneas de código, medir el esfuerzo

Principales conclusiones

- Las pruebas ayudan a elevar la calidad del software, previniendo que los defectos pasen a los usuarios finales.
- Las pruebas no son gratuitas, se debe dedicar un esfuerzo considerable a la implementación de un proceso de pruebas.
- El proceso de pruebas, no es trivial. Debe planificarse, controlarse y asignar a él recursos altamente calificados.
- Cuanto antes se detecte un defecto, más barato resultará su corrección. Cuanto más tarde se detecte, más caro.
- La mejor aproximación a una estrategia de pruebas, es aquella que permite la superposición de éstas con las actividades específicas de desarrollo.

Defectos: Cuantos son, con qué severidad, tipos de defectos, cuantos se corrigieron y no, cubrimiento de sentencias, cuantos casos de prueba.